CS2102

AY25/26 Sem 1

by ngmh

DBMS

Transaction

- Finite sequence of database operations, constituting smallest logical unit of work from application perspective
- Properties:
  - Atomicity: Either all effects are reflected or none
  - Consistency: Guaranteed to yield correct database state
  - Isolation: Transaction is isolated from effect of concurrent transactions
  - Durability: Effects of commit are permanent even in case of failures
- Problems with Concurrent Transactions: Lost updates (Effect of transaction overwritten), Dirty reads (Reading invalid data), Unrepeatable read (Value retrieved twice with differing values)
- Serialisability: A concurrent execution of a set of transactions is serialisable if its execution is equivalent to some serial execution of the same set of transactions
- Two executions are equivalent if they have the same effect on the data
- DBMS supports concurrent executions for performance and enforces serialisability of concurrent executions for data integrity

Architecture

- 3-Tier Architecture / Levels of Data Abstraction
- External Schema: User or group-specific view on data
- Logical Schema: Logical organisation of data forming a data model, unified representation of all data, supports logical data independence and physical data independence
- Physical Schema: Organisation of data on disk and in memory, database as collections of fields, arrays, records, files, pages, etc.
- Logical Data Independence: Ability to change logical schema without affecting external schemas
- Physical Data Independence: Representation of data being independent from physical schema, can be changed without affecting logical schema
- Data Model: Set concepts for describing data
- Schema: Description of structure of DB using data model concepts
- Schema Instance: Content of a DB at a particular time

Relational Model

- Based on relations; tables with rows and columns
- Relation Schema: Specifies attributes (columns) and data constraints (domain constraints etc.)
- Domain: Set of atomic values
- Domain of attribute A_i is $dom(A_i)$, and each value v is either $v \in dom(A_i) \vee v = null$ (unknown or unspecified)
- Relation: Set of tuples, represented by relation schema $R(A_1, A_2, ..., A_n)$
- Each instance of a schema R is $\subseteq \{(a_1, a_2, ..., a_n) | a_i \in dom(A_i) \cup \{NULL\}\}$
- $R.A_i$ is attribute A_i of relation R
- Relational Database Schema: Set of relation schemas + data constraints
- Relational Database: Collection of tables
- Integrity Constraints: Conditions that restrict what constitutes valid data, checked by DBMS
- Structural Constraints: Domain constraints, Key constraints, Foreign Key constraints
- Structural: Inherent to data model, not dependent on application
- General Constraints: Dependent on specific application etc. triggers

Key Constraints

- Superkey: Subset of attributes that uniquely identifies a tuple in a relation
- Key: Superkey that is also minimal (No proper subset is a superkey)
- Every relation has at least 1 superkey
- Candidate Keys: Set of all keys for a relation
- Prime Attribute: Attribute of candidate key
- Primary Key: Selected candidate key for a relation, with added non-null constraint (entity integrity constraint), underlined in schema notation
- Foreign Key: Subset of attributes of relation A that refers to primary key of relation B
- Each foreign key in A must either be a null value or appear as a primary key in B
- Foreign keys can be self-referential

Considerations

- Structural integrity constraints do not cover application-independent constraints
- Integrity constraints are not mandatory
- Performance might decrease due to additional checking of constraints, could improve due to creation of indexes which speed up retrieval

SQL

- Structured Query Language
- Domain-specific and declarative language
- Interactive SQL: Directly writing SQL statements to an interface
- Non-interactive SQL: Application written in host language with interfaces
- Statement Level Interface: Mixture of host language and SQL statements
- Call Level Interface: Entirely written in host language and SQL statements are executed with function calls
- Interactive SQL: Directly writing SQL statements into interface such as Command Line or Graphical User Interfaces

Commands

- DDL: Data Definition (CREATE, ALTER, DROP, RENAME, TRUNCATE)
- DML: Data Manipulation (INSERT, UPDATE, DELETE, MERGE)
- DQL: Data Query (SELECT)
- DCL: Data Control (GRANT, REVOKE)
- TCL: Transaction Control (BEGIN, COMMIT, ROLLBACK, SAVEPOINT)

Create / Insert / Delete / Update

- Create:

```
CREATE TABLE Employees (
    id INTEGER,
    name VARCHAR(50),
    age INTEGER,
    role VARCHAR(50)
)
```

- Basic Data Types: BOOLEAN, INTEGER, FLOAT8, NUMERIC [(p, s)], CHAR(n), VARCHAR(n), TEXT, DATE, TIMESTAMP...
- Insert Into:

```
INSERT INTO Employees VALUES (101, 'Sarah', 25, 'dev');
```

- Delete:

```
DELETE FROM Employees WHERE role = 'dev';
```

- Update:

```
UPDATE Employees SET age = age + 1 WHERE name = 'Sarah';
```

Three-Valued Logic

- Result of comparing with null is unknown, while result of arithmetic with null is null
- Values: True, False, Unknown
- Use IS NULL to check if null value
- Use IS DISTINCT FROM to check inequality, equivalent to <> if non-null
- Conjunction: False > null > True
- Disjunction: True > null > False

Integrity Constraints

- Types of Constraints: Not-null, Unique, Primary key, Foreign key, General
- Column Constraint: Applies to a single column, specified at column definition
- Table Constraint: Applies to one or more columns, specified after column definitions
- Assertion: Stand-alone command
- Not-Null: Violated when there is a tuple where "id IS NOT NULL" evaluates to false

```
id vARCHAR(50) CONSTRAINT nn_id NOT NULL,
```

- Unique: Violated when for any two tuples, "(id1 <> id2) or (pname1 <> pname2)" returns false

```
id INTEGER CONSTRAINT u_id UNIQUE,
CONSTRAINT u_alloc UNIQUE (eid, pname)
```

- Primary Key: Selected key uniquely identifying tuples, Prime attributes cannot be null

```
id INTEGER PRIMARY KEY,
PRIMARY KEY (eid, pname)
```

- Foreign Key: Subset of attributes of A refer to primary key of B, violated when foreign key does not appear as primary key in referenced relation and is not null

```
FOREIGN KEY (eid) REFERENCES Employees (id),
```

- Behaviour can be extended with ON DELETE / UPDATE <action>
  - NO ACTION: rejects if it violates constraint
  - RESTRICT: similar, but can defer check of constraint
  - CASCADE: propagates to referencing tuples
  - SET DEFAULT: update foreign key to some default value
  - SET NULL: update foreign key to null
- CHECK constraint: Most basic general constraint, specifies Boolean expression that column values must satisfy

```
CONSTRAINT valid_lifetime CHECK (start_year <= end_year)
```

- Assertion: CREATE ASSERTION, used for arbitrary constraints, cannot modify data, and not linked to specific table, most RDBMS do not support
- All constraints can be specified named or unnamed
- All column constraints can be specified as table constraints other than not null
- Table constraints referring to a single column can be specified as a column constraint
- Column and table constraints can be combined

Deferrable Constraints

- By default, constraints are check immediately, and violations will cause rollbacks
- Can defer constraints to end of transaction
- Allows cyclic foreign key constraints
- No need to care about order of statements, and performance boost when constraint checks are bottleneck
- Troubleshooting is harder, data definition might be ambiguous, and queries might incur performance penalties when certain checks occur at run time

```
CONSTRAINT manager FOREIGN KEY (manager)
```

```
REFERENCES Employees (id)
DEFERRABLE INITIALLY IMMEDIATE/DEFERRED
...
SET CONSTRAINT manager DEFERRED;
```

Modifying Schema

- Alter Table: e.g. Add/Drop Columns/Constraints, change table specifications

```
ALTER TABLE Projects ALTER COLUMN start_year DROP DEFAULT;
ALTER TABLE Projects ADD COLUMN budge NUMERIC DEFAULT 0.0;
ALTER TABLE Teams ADD CONSTRAINT eid_fkey FOREIGN KEY (eid)
REFERENCES Employees (id);
```

- Drop Table: Without dependent objects, check if exists first; With dependent objects choose to cascade

```
DROP TABLE IF EXISTS Projects;
DROP TABLE Projects CASCADE;
```

Entity Relationship Model

- Data is described using entities and their relationships, with information described as attributes
- Data constraints can also be modeled
- Entity: Real world objects that are distinguishable from others
- Entity Set: Collection of entities of the same type, represented by rectangles, typically nouns
- Attribute: Specific information describing an entity, represented by a small circle
- Key Attributes: Uniquely identify entity, indicated by filled circle
- Derived Attribute: Derived from other attributes, indicated by dashed line
- Composite Key Attributes: Attributes which together uniquely identify each entity, indicated by additional connecting line
- Composite Attributes: Prefer splitting it up into components
- Multivalued Attributes: Refer to a set or list of values, can either be single valued attributes or dedicated entity set

Relationship Sets

- Relationship: Association between two or more entities
- Relationship Set: Collection of relationships of the same type, represented by diamonds, can have their own attributes, typically verbs
- Role: Descriptor of entity set's participation in relationship
- Degree: How many entity roles participate in a relationship (2 and 3 most common)

Relationship Constraints

- Cardinality: Upper bound of how many times entity can participate in relationship
- Many-Many, Many-One, One-One
- Participation: Lower bound to how many times entity can participate in relationship
- Indicated by (min, max) labels: User (0, n) makes (1, 1) Booking means each user can make multiple bookings, but each booking is only made by exactly one user

Dependency Constraints

- Weak entity set: Entity set without its own key, can only be uniquely indentified by considering primary key of owner entity
- Identified by double-lined rectangles or diamonds
- Many to one relationship from weak entity set to owner entity set, must have (1, 1) attachment to identifying relationship
- Has partial key which uniquely identifies weak entity with context of owner entity

Aggregation

- Relationships between entity sets and relationship sets
- Treat relationships as higher-level entities, indicated by rectangle around them

Relational Mapping

- Mapping from ER model to relational schema
- Straightforward mapping from entity sets to tables
- Multivalued attributes: With fixed number of single-valued attributes, can decompose, if not create a new entity set with foreign key
- Relationship sets: Model in new table, with key attributes of all participating entity sets as well as own relationship attributes
- Use primary and foreign keys to capture cardinality and participation constraints
- Weak entity sets: Need foreign keys an ON DELETE/UPDATE
- Aggregation: Primary key of aggregation relationship, associated entity set, and descriptive attributes
- ER diagram should capture as many constraints as possible without imposing unnecessary ones, similarly for relational schema

More SQL

- Multiset / bag semantics (relational models are sets)
- Query is done with SELECT statement
- Basic Form:

```
SELECT [DISTINCT] target_list FROM relation_list
[WHERE conditions]
```

Simple Queries

- Use wildcard "*" to include all attributes
- Use "expr BETWEEN x AND Y" for basic value range conditions
- Use "AS" to rename attributes
- Use "DISTINCT" to eliminate duplicates; IS DISTINCT FROM must evaluate to true for at least one attribute
- WHERE clause imposes conditions
- "_" matches single character, "%" matches zero or more, can use regexs as well

Set Operations

- Union-Compatible: R and S have the same number of attributes, and corresponding attributes have same or compatible domains without implicit conversions
- UNION, INTERSECT, EXCEPT (Set difference) can be used to combine results; use ALL to not eliminate duplicates
- Column names follow left relation

Join Queries

- Cartesian Join: Return all possible pairs across relations
- Join: Cross product + attribute selection
- Inner Join: Cartesian Join + Selection Condition + Attribute Selection
- Natural Join: Join condition implied by attribute names, only defined for equality comparison, result only contains one instance of matching attributes, performed over al attributes, equivalent to Cartesian Join if there are no common attributes
- Outer Join: Inner Join but also returns dangling tuples which might not have matches
- LEFT OUTER JOIN = INNER JOIN + all remaining from left table
- RIGHT OUTER JOIN = INNER JOIN + all remaining from right table
- FULL OUTER JOIN = INNER JOIN + all remaining from both tables

Subqueries

- In FROM clause: Consequence of closure property (result is also a table), must be enclosed in parentheses, table alias mandatory, column aliases optional
- Expressions: IN, EXISTS, ANY/SOME, ALL
- IN: Subquery must return exactly one column, can typically be replaced with inner join, NOT IN can typically be replaced with outer join, can manually specify subquery result
- ANY/SOME: expr op ANY (subquery), subquery must return exactly one column, expr is compared to each row with op
- ALL: Similar, but must be true for all subquery rows
- Correlated Subquery: Uses value from outer query, could lead to slow performance, if subquery has null value, ALL condition can never evaluate to true
- Naming ambiguities can cause problems, resolve using table aliases
- Scoping Rules: Table alias declared in subquery can only be used in it or its own subqueries
- If same table is declared in both a subquery and outside the innermost one is applied
- EXISTS: Checks is subquery returns at least one tuple
- Scalar Subquery: Subquery returns a single value, can be used as expression in queries
- Row Constructors: Allow subqueries to return more than one attribute or column, must match subquery number of attributes

Sorting and Rank Based Selection

- Use ORDER BY with ASC or DESC
- Can sort by multiple attributes and with different orders in order
- LIMIT returns a portion of the table
- OFFSET specifies position of first tuple to be considered

More More SQL

Common SQL Constructs

Aggregation

- Compute a single value from a set of tuples
- e.g. MIN, MAX, AVG, COUNT, SUM
- null might be treated differently
- Data type affects applicability and return data type

Grouping

- Use GROUP BY to partition relation into groups
- Aggregation functions are applied over groups instead
- Two tuples belong to the same group if for all involved pairs of attributes it IS NOT DISTINCT
- If A_i of R appears in SELECT, it must either appear in GROUP BY or appear as input of an aggregation function in SELECT
- Or satisfy global condition where primary key of R appears in GROUP BY
- HAVING: Conditions check for each group defined by a GROUP BY clause, typically involve aggregate functions
- Must appear in GROUP BY clause, or as an input of the aggregation function for itself, or primary key of R appears in GROUP BY clause
- Evaluation: FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT/OFFSET

Conditional Expression

- CASE: Similar to switch statement, list of WHEN THEN with ELSE base case, only one result to be returned
- COALESCE: Returns first non-NULL value in list of input arguments
- NULLIF: Returns NULL if both values are equal, if not returns first value

Structuring Queries

- Common Table Expression: Temporary named query, allowing structure for queries to improve readability
- WITH ctename AS ctebody ...
- View / Virtual Table: Permanently named query, result of a query is not permanently stored, can be used like normal tables
- Use CREATE VIEW AS ... to make a VIEW
- Can be used almost like a normal table, with restrictions for INSERT, UPDATE, DELETE
- Updatable View: Only one entry in FROM clause, no WITH, DISTINCT, GROUP BY, HAVING, LIMIT, OFFSET, UNION, INTERSECT, EXCEPT, ALL, aggregate functions

Extended Concepts

- SQL only supports existential quantification (EXISTS)
- Universal Quantification: Transform query to EXISTS using logical equivalences
- Recursive Queries: Each query for X stops requires X joins
- Use CTEs for recursive queries

```
WITH RECURSIVE cte_name AS (
    Q1
    UNION [ALL]
    Q2 (cte_name)
)
SELECT * FROM cte_name;
WITH RECURSIVE flight_path AS (
    SELECT from_code, to_code, 0, AS stops
    FROM connections
    WHERE from_code = 'SIN'
    UNION ALL
    SELECT c.from_code, c.to_code, p.stops + 1
    FROM flight_path p, connections c
    WHERE p.to_code = c.from_code
    AND p.stops < 2
)
SELECT DISTINCT to_code, stops
FROM flight_path
ORDER BY stops ASC;
```